

Lab 12 - Cloud-Edge Architecture Design and Measured Trade-offs

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 05: Evaluating Cloud and Edge AI Creative Workflows Outcome: ELO6 | In-class time: 60 minutes of scheduled practical time

Competency	Statement
Ability A7	Design the architecture of applied vision systems
Ability A8	Design, develop and evaluate edge-based and cloud-based systems
Knowledge K11	Vision system architecture
Knowledge K12	Vision communication protocols
Knowledge K13	Real-world design constraints and solution options

Scenario

NorthQuay wants the whole creative pipeline running across 41 stores and one cloud account. Measure the real per-stage latency of the representative local operations drawn from Labs 02-11 on this machine, compute the payload that would cross each candidate split point, choose the protocols, and defend one split with numbers rather than preference.

What you will produce

out/stage-latency.csv (measured, not estimated), out/bandwidth-model.csv for four upload strategies at 41 stores, out/split-decision.md naming the chosen split point with its latency, bandwidth, privacy and cost consequences, and a labelled four-layer architecture diagram.

Tools: Python 3 | opencv-python | NumPy | offline with prepared dependencies; no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Measure per-stage latency on this machine
2. Model the payload at each split point
3. Compute bandwidth and cost at 41 stores
4. Select protocols per link
5. Defend one split with the numbers

Files in this folder

Path	What it is
reference/nq-pipeline-clip.mp4	Synthetic 10-second 1280x720 25 fps clip used as the pipeline input for timing (deterministically rendered with OpenCV - SIMULATED, not generative-model output)
data/deployment-constraints.json	Store count, link speeds, privacy classification, latency budget per use case, and the indicative unit costs used for the cost model
data/protocol-catalogue.json	Candidate protocols with their payload class, typical latency class and transport, for the protocol-selection step
data/split-candidates.json	The four candidate split points, each with the stage boundary it cuts
pipeline_bench.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Work inside this lab folder with a prepared Python/OpenCV/NumPy environment; create out/. This local benchmark measures representative classical stages, not every previous lab or a deployed cloud system. Assume one camera per store unless cameras_per_store is explicitly set. Read split-candidates.json and map its boundaries to the four payload models; a split is a hypothetical architecture, not an executable cloud deployment. Record all additional assumptions in out/split-decision.md.
2. Read data/deployment-constraints.json. Note the four numbers that will drive every decision: 41 stores, the per-store uplink in Mbit/s, the latency budget for the shelf-alert use case, and the privacy classification of the camera feed.
3. Draw the four-layer architecture before measuring anything. Layer 1 IoT SENSING: the store cameras and the catalogue capture rig. Layer 2 EDGE INTELLIGENCE: pre-processing, lightweight inference and any federated update. Layer 3 GENERATIVE VISUAL MODELLING: the generation and heavy editing stages, wherever they run. Layer 4 APPLICATION: the review queue, the asset store and the dashboards.
4. Mark on your diagram which of your Labs 02-11 stages sits in which layer. Decode, colour convert and background subtraction are layer 2; detection and tracking are layer 2 or 3 depending on model size; generation, inpainting and outpainting are layer 3; the consistency gate and the review queue are layer 4.
5. Now measure. Run the timing harness, which executes the real pipeline stages on the supplied clip and reports per-stage milliseconds per frame over the whole clip, with a median and a 95th percentile rather than a single average.

```
python3 pipeline_bench.py --video reference/nq-pipeline-clip.mp4 --constraints data/
deployment-constraints.json --out out
```

6. Read out/stage-latency.csv. Identify the single most expensive stage. Note that the expensive stage on this machine may not be the expensive stage on a store device - record the CPU this was measured on, because a latency number without its hardware is not a measurement.
7. Read the directly timed harness end-to-end median/p95 row; the sum of isolated stage medians is only a rough estimate and is not an end-to-end median, and compare it against the shelf-alert latency budget. State whether the budget is met on this hardware, and by what margin.
8. Model the payload at each split point. Split A sends RAW FRAMES to the cloud: $1280 \times 720 \times 3 \text{ bytes} \times 25 \text{ fps} = 69.1 \text{ MB/s}$ uncompressed per camera. Split B sends ENCODED VIDEO. Split C sends FEATURE VECTORS. Split D sends EVENTS ONLY - a JSON record when something happens.
9. Compute each payload honestly. For split C use the descriptor sizes you measured in Lab 06: an ORB descriptor is 32 bytes and a default float32 SIFT descriptor is 512 bytes; keypoint coordinates/scale/orientation and framing add overhead, so 500 ORB keypoints per frame is 16 kB per frame, or 400 kB/s at 25 fps. For split D assume the event rate in the constraints file, at roughly 400 bytes per event.
10. Scale to 41 stores and convert to the units the network team uses. Report each split in Mbit/s per store and aggregate Gbit/s across the estate, then compare against the per-store uplink. Check whether any split is arithmetically impossible on the stated uplink - identify it and say so.
11. Compute the monthly transferred data volume for each split using the operating hours in the constraints file, and apply the indicative per-GB cost. Label this clearly as an ILLUSTRATIVE cost model built on the stated unit rates, not a quotation.
12. Select protocols per link from data/protocol-catalogue.json. Camera to edge node: RTSP or WebRTC for a live stream. Edge node to cloud for telemetry and events: MQTT (publish/subscribe) or CoAP (request/response, with optional Observe) for constrained links. Edge or application to a model endpoint: HTTPS/REST or gRPC. Rendered assets to storage: object storage over HTTPS.
13. Justify each protocol choice by PAYLOAD CLASS and LATENCY CLASS, not by familiarity. Write one line per link: 'link, payload class, latency class, protocol, reason.' A protocol chosen without reference to payload size is a guess.
14. Apply the generative-latency model. For fixed architecture, resolution and hardware, approximate sampling work as proportional to the number of denoising steps S ; video sampling latency scales with $S \times F$ for F frames. Using the step counts in the constraints file, compute the relative cost of a 25-step image against a 25-step, 200-frame clip - this simplified ratio alone cannot determine whether video generation can run at the edge in this design.
15. The S times F model assumes constant per-frame step cost and ignores temporal attention, batching and hardware effects. Read generation-model.csv as an illustrative scaling scenario, not a latency prediction. Weight memory excludes activations, caches, runtime and quantisation metadata. Feature payload excludes metadata unless metadata_bytes_per_keypoint is supplied; sending descriptors is not automatically privacy-preserving. Store-to-cloud transfer is not necessarily billed as cloud egress; verify direction and unit-rate basis before any real quote.
16. Apply the edge-fit levers. Compute the parameter memory of the stated model at 32-bit and at 8-bit: memory scales by $b/32$, so 8-bit is a quarter. Then compute the trainable parameter count for a low-rank adapter of rank r on a $d \times m$ layer: $r(d+m)$ against $d \times m$. Put both numbers in your decision.
17. Estimate energy per sample as overhead factor $\times$ energy per FLOP $\times$ FLOPs per sample using the values in the constraints file, and state what you would have to report for a real carbon figure: hardware inventory, total kWh, the overhead factor, grid carbon intensity and joules per sample. Mark your figure ILLUSTRATIVE.
18. Apply the deployment decision matrix to each eligible selected use case from Lab 01: low-latency image work suggests a distilled sampler with INT8 weights; an edge device suggests INT8 plus parameter-efficient adapters; high resolution suggests latent-space modelling with a few-step sampler; short video suggests temporal attention with

distilled steps; brand and style control suggests retrieval conditioning with filters; and a tight privacy constraint suggests a local open model with provenance tracking.

19. Handle privacy explicitly. The camera feed is classified in the constraints file. State which split points are ELIMINATED by that classification before any latency or cost argument is made - a privacy constraint is a gate, not a weighting.

20. Write out/split-decision.md. Name your chosen split point. Give its measured local-harness latency (not split-specific remote latency), its computed bandwidth per store and across the estate, its illustrative monthly cost, the protocols on each link, and the privacy argument. Then name the runner-up and state the single number that decided between them.

21. Write the residual-risk section. Name at least three: the measurement was taken on developer hardware and not on the store device; the event rate is an assumption; and the model may be replaced, which changes the latency and memory terms. State how each would be closed before build.

22. Save out/stage-latency.csv, out/bandwidth-model.csv, out/split-decision.md and your labelled architecture diagram, then complete the evidence checklist.

Verify

out/stage-latency.csv contains measured median and p95 milliseconds for every pipeline stage with the CPU recorded, out/bandwidth-model.csv gives Mbit/s per store and Gbit/s across 41 stores for all four splits and flags any that exceed the stated uplink, and out/split-decision.md names one split with its latency, bandwidth, cost, protocols, privacy argument and the deciding number against the runner-up.

Expected outputs

- Report latency separately from throughput: compare measured serial local-harness milliseconds/frame with 1000/FPS. Even if a frame meets the alert latency budget, processing slower than its arrival interval causes queue growth unless frames are dropped, processing is parallelised or stages are reduced. Results are hardware-dependent.
- out/stage-latency.csv - one row per stage: median ms, p95 ms, frames measured, plus the CPU/architecture string and OpenCV version in environment.json; generation-model.csv is a separate assumed-cost table.
- A directly timed local-harness end-to-end median/p95 against the budget, excluding network/cloud/queue delays.
- out/bandwidth-model.csv - four splits x (bytes/s per camera, Mbit/s per store, Gbit/s at 41 stores, monthly GB, illustrative monthly cost, feasible on uplink).
- Raw-frame split feasibility calculated from the actual input and stated uplink, not forced.
- out/split-decision.md with the chosen split, the runner-up and the deciding number.
- A labelled four-layer architecture diagram with every Lab 02-11 stage placed.
- The INT8 memory ratio (0.25) and the LoRA trainable-parameter comparison.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- [] Measured service time versus frame-arrival interval (1000/FPS), the implied serial throughput 1000/service_ms, and a queue-control plan if arrival rate exceeds processing capacity; no sustained-rate claim from latency alone.
- [] out/stage-latency.csv with the CPU and OpenCV version recorded.
- [] The end-to-end median latency and the margin against the budget.
- [] out/bandwidth-model.csv with the infeasible split identified.
- [] The protocol table: link, payload class, latency class, protocol, reason.

- [] The S versus S x F generative latency ratio you computed.
- [] The INT8 and low-rank-adaptor arithmetic.
- [] The privacy gate statement naming which splits it eliminates.
- [] out/split-decision.md including the runner-up and the deciding number.
- [] Three residual risks with how each would be closed.
- [] An explicit ILLUSTRATIVE label on the cost and energy figures.

Troubleshooting

Symptom	What to do
Stage latencies vary wildly between runs	Possible causes include warm-up, OS load, scheduling, frequency changes or thermal limits. The harness discards the first five frames and reports a median and p95 for exactly this reason - report those, not a mean, and say how many frames were measured.
The generation stage cannot be timed because no model runs locally	Correct, and expected. Time the stages you actually run, and model the generation stage from the documented S and S x F relationship instead. Label it MODELLED, not MEASURED - mixing the two in one table is the mistake this lab exists to prevent.
Bandwidth numbers look absurdly large	Check your bits-versus-bytes conversion: multiply bytes per second by 8 to get bits per second, then divide by 1e6 for Mbit/s. A factor-of-eight error here changes the architecture.
Every split looks feasible	Check the input dimensions, frame rate, camera count and link budget; all may legitimately fit a small fixture. Split A is defined as uncompressed frames - 1280 x 720 x 3 x 25 bytes per second per camera. That is the point of including it.
The cost model produces a number you want to quote	Do not. It is built on indicative unit rates in a classroom constraints file. Label it ILLUSTRATIVE everywhere it appears, and state that a real figure requires current vendor pricing and a measured event rate.
You cannot choose between two splits	That usually means you have not applied the privacy gate. Eliminate on constraint first, then optimise on cost among what survives.

References used in this lab

- [S13] Tschochohei and Schenker (2026), as above - *Ch.9 pp.150-152*.

Multimodal prompt injection, steganographic payloads in images, side-channel attacks, and the Samsung data-leak example used to motivate data-protection-aware architecture choices.

- [S15] Mewada et al. (2026), as above - *Ch.3 pp.35-44 (Vishnoi, Sisodia, Khare and Yadav, 'Sensors-to-synthesis: Edge AI and IoT for Generative Visual Systems')*.

The four-layer sensors-to-synthesis architecture (IoT sensing / edge intelligence / generative visual modelling / application) shown in Fig. 3.1; MQTT and CoAP as the lightweight sensor-to-edge protocols; split computing between edge and cloud; federated learning; Table 3.2 comparing VAE, GAN and diffusion for edge suitability; Table 3.3 domain applications; Table 3.4 challenges and future directions (compute/energy, latency, privacy/security, scalability/interoperability, ethics; pruning, quantisation, knowledge distillation, neuromorphic and

FPGA accelerators, differential privacy, standardised APIs, containerised deployment).

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD², LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S18] Mewada et al. (2026), as above - *Ch.7 pp.98-100*.

Peak memory = parameter memory + activations (+ optimiser state during training); diffusion sampling latency proportional to S denoising steps for images and to S x F for video; moving weights from 32-bit to b-bit scales parameter memory by b/32; low-rank adapters train r(d+m) parameters versus d*m; energy per sample approximated by overhead factor x energy-per-FLOP x FLOPs-per-sample, with hardware, kWh, PUE, grid carbon intensity and joules-per-sample all reportable; Table 7.4 deployment decision matrix (low-latency image -> distilled sampler + INT8; edge device -> INT8 + PEFT adapters; high resolution -> latent diffusion + few-step sampler; short video -> temporal attention + distilled steps; brand/style control -> retrieval conditioning + filters; tight privacy -> local open model + provenance tracking).

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.